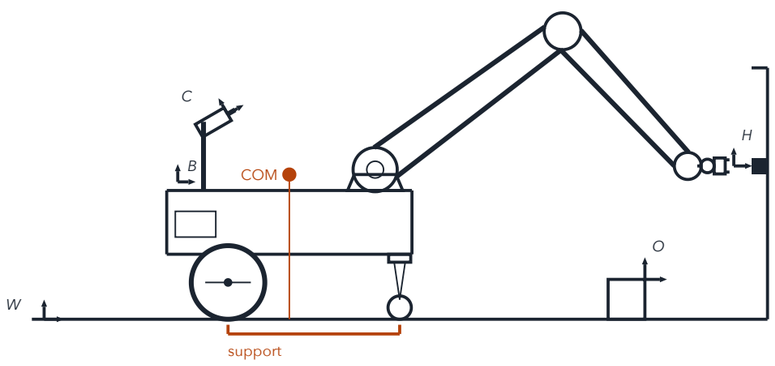
Mobile Manipulator Learning Map

From basic Linux and Python to a two-wheeled robot arm that presses a button and picks up an object.

Prepare · Build the classical robot · Add learning · Mission · Safety and limits

Follow the numbers. Do not move on until you meet Pass. Build in simulation first, then on the real robot.

One robot, one idea



$${}^{W}T_{H} = {}^{W}T_{B}(x, y, \theta) \cdot {}^{B}T_{H}(q)$$

Hand in world = base in world × hand relative to base (q = joint angles).

$${}^{B}T_{O} = {}^{B}T_{C} \cdot {}^{C}T_{O}$$

Target from base = camera from base (calibration) × target from camera (vision).

Wheel odometry drifts. So park, look again, and act from the base frame.

T = pose of one frame in another. W world · B base · C camera · H hand · O target.

Reference robot: two driven wheels and a passive caster, so it stands still when powered off.

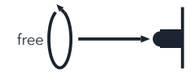
No sideways motion



$v = r(\omega_R + \omega_L)/2 \qquad \omega = r(\omega_R - \omega_L)/b$

Wheels roll, they do not slide. Moving 5 cm left takes a manoeuvre, so choose your parking pose carefully.
r wheel radius · b track width

Count the constraints



press = 3 position + 2 axis = 5

A 5-joint arm has no spare joint for this. Turning the base adds one. Counting is necessary, not sufficient.

Keep the COM inside



Stable: COM ground point inside the support polygon.

Reaching and braking shrink the margin. Keep batteries low and stow the arm before driving.

Friction holds it



$2\,\mu\,F_n \geq m\,g$

Squeeze force and surface friction must carry the weight. Soft, grippy fingertips help most.
μ friction coefficient · F_n grip force

The route

Before real motors: e-stop in reach, speed and joint limits, command watchdog, arm stowed while driving.

	0 Toolbelt	1 Describe	2 Drive	3 Reach	4 See	5 Press	6 Pick	7 Learn	8 Mission
Time	2 weeks	2 weeks	2 weeks	3 weeks	2 weeks	2 weeks	3 weeks	4 weeks	2 weeks
Learn	Git, Python environments, NumPy rotations and transforms, ROS 2 nodes and topics.	URDF robot models, the TF frame tree, RViz, mass and centre of mass.	Differential drive, encoders, PID wheel control, odometry, ros2_control.	Forward and inverse kinematics, joint limits, trajectories, MoveIt 2.	Camera model, calibration, AprilTag poses, hand–eye calibration, OpenCV.	Parking pose, slow approach, contact detection, force limits.	Grasp poses, friction, object detection, state machines, recovery on failure.	Teleoperation, demonstration data, ACT and Diffusion Policy, LeRobot, PyTorch.	System integration, behaviour trees, rosbag2 logs, failure analysis.
Build	Two ROS 2 nodes that talk. A transform library in NumPy.	A simulated model of your robot with masses.	Velocity-controlled base, in simulation and then real.	Arm that moves its tip to commanded points.	Pipeline that gives a target pose in the base frame.	First task: drive to a panel and press a button.	Second task: find, grasp and lift one object.	A policy trained from your own demonstrations for the pick.	Drive, press, pick and return as one program.
Pass	**Your composed transforms match a hand calculation.**	**Full TF tree. COM stays inside support in every planned arm pose.**	**Drive a 1 m square 10 times. Report the closing error.**	**Touch 5 marked points, 10 times each. Report mean and worst error.**	**Point at a tag in 5 positions. Report the error in mm.**	**From 1 m away: at least 16 of 20 presses, no tipping.**	**At least 14 of 20 picks, with failures logged by cause.**	**Compare with stop 6 on the same 20 trials. Either result is valid if you can explain it.**	**10 logged missions with a failure breakdown, and a demo.**

GPU needed only at stop 7. LeRobot usually needs a thin bridge to ROS 2.

Report k of n, not a percentage. 8 of 10 is compatible with a true rate of 49–94% (95% Wilson interval), so use 20 or more trials to compare.

How it fits

What runs on the robot

Sense → **Estimate** → **Decide** → **Act** → **Physics**

camera · wheel encoders · joint encoders · servo load → odometry · tag pose · TF frames → state machine · planner · policy → wheel PID · joint control · gripper → gravity · slip · contact

Typical rates: control 50–100 Hz · camera 30 Hz · policy 10–30 Hz · task about 1 Hz

Safety overrides everything.

How you improve it

Model → Simulate → Run on robot → Log → Measure → Change one thing

Five ways to decide

Method	You supply	Breaks when	Stop
Scripted state machine	Steps and thresholds	The situation was not planned for	5–6
Model-based planning	Robot and scene geometry	The model or sensing is wrong	3–6
Imitation learning	Human demonstrations	The scene differs from the demos	7
Reinforcement learning	Simulator and reward	The reward or simulator misleads	Next
Foundation models (VLA)	Pretrained vision-language-action model, your data	Too slow, or failures are hard to explain	Next

After the mission

- Reach while moving: whole-body control
- Self-balance on two wheels: inverted pendulum, IMU, LQR
- Map the room: SLAM and Nav2
- Train in simulation: RL with MuJoCo Playground or Isaac Lab (GPU)
- Take instructions: foundation models such as SmolVLA

Times are planning estimates at about 8 hours per week, about 22 weeks in total, not measurements. Pass targets are starter values: tune them to your hardware and justify the change.

Reference kit, checked Nov 2026: SO-101 arm (5 joints + gripper) · 2 geared DC motors with encoders · caster · single-board computer · camera · Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic. One option, not an endorsement. Check driver support before choosing a newer ROS 2.